

Day 4

Threat Intelligence · 위험도 점수화

통합 운영자료 (개념+실습 병행) · 6/11 木 · 09:00–17:00

내부 발견에 외부 위협 정보를 결합한다. CVE·KEV·EPSS로 발견을 우선순위로 바꾼다.

이 파일 하나로 하루 수업을 끝까지 진행합니다. 상세 이혼은 *Appendix*를 참조하세요.

Day 4 운영 흐름

내부 발간에 외부 위협 정보를 결합한다. CVE·KEV·EPSS로 발간을 우선순위로 바꾼다.

09:00-10:20	10:20 -	80m	개념+파일
11:40-13:00	13:00 -		
14:20-15:40	15:40 -	80m	점수화 확인
	17:00		

Day 4 — 무엇을 배우고 무엇을 만드는가

학습 목표·실습 목표·사용 파일·성공 기준을 시작 전에 꼭 봐야 합니다.

학습 목표 (개념·실습 병행)

- Threat Intelligence의 역할
- CVE·KEV·EPSS 세 신호의 차이
- 위험도 점수화 공식의 의미

실습 목표 (실습·산출물 확인)

- threat_tools.py 3함수 구조 확인
- threat_agent.py 실행
- 위험도 점수 가중치 조정
- KEV·EPSS가 우선순위에 미치는 영향 관찰

사용 파일

- agents/threat_agent.py
- templates/threat_tools.py
- agents/report_agent.py

성공 기준

- threat_context.json 생성 확인
- KEV 등재 시 우선순위 상승 확인
- risk_score_table.md 정렬 재현

5일 로드맵: 단일 Agent에서 통합 Security PoC까지

오늘이 전체 과정에서 어디에 위치하는지 먼저 확인합니다.

단일 Agent에서 통합 Security PoC까지 이어지는 하나의 파이프라인입니다.



1 개념 설명



2 파일 확인



3 직접 실행



4 산출물 확인



5 관찰 정리



6 다음 연결

개념 + 실습 병행 운영 패턴 (09:00-17:00)

개념을 설명하고 바로 파일 확인·실행·산출물 검증으로 이어갑니다.

수업은 이론을 몰아서 듣는 방식이 아니라, 작은 학습 루프를 하루에 3회 이상 반복합니다.



매일 최소 3회 이상 직접 실행



운영 메모: DDD 관점: 차량 제어 모듈 같은 도메인 맥락을 위험도 계산에 반영한다.

오늘의 자료 구조: PPT → 코드 → 명령 → 산출물

수강생이 지금 보는 장표가 어떤 파일과 어떤 결과물로 이어지는지 명확히 보여줍니다.

PPT의 개념은 실제 Codespaces 파일·명령어·산출물로 바로 연결되어야 합니다.



PPT 개념

위협정보를 결합한
Agent

CVE · KEV · EPSS

확장 개념
Risk Scoring, Prioritization,
Domain Context



관련 파일

templates/threat_tools.py

agents/threat_agent.py

agents/report_agent.py



실행 명령

```
python agents/threat_agent.py
```

```
python orchestrator/supervisor_graph.py --target  
sample_app/ --threat-intel
```



산출물

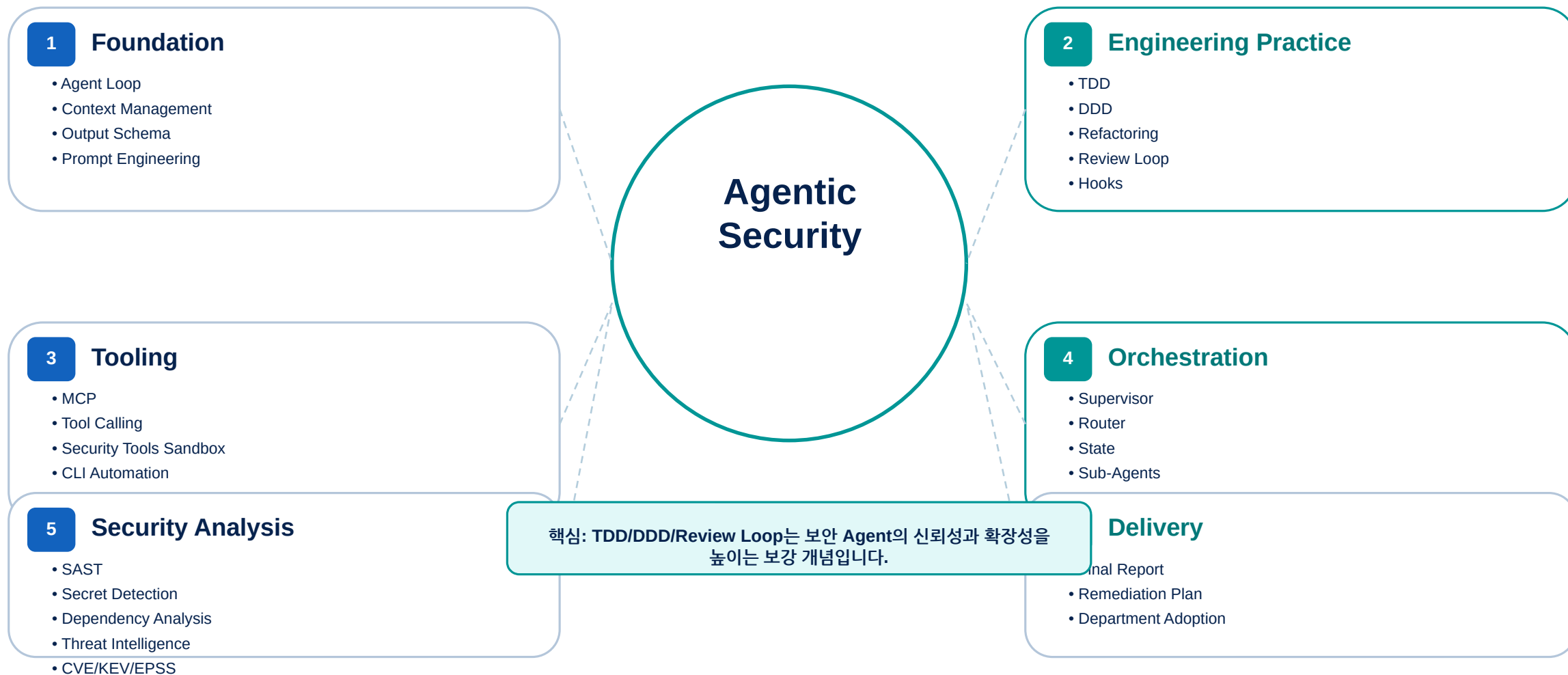
reports/threat_context.json

reports/risk_score_table.md

보강 개념 맵: 실전 에이전틱 보안 엔지니어링

TDD·DDD·Review Loop·Hooks 등은 현재 실습 흐름을 보강하는 개념으로 연결합니다.

레퍼런스의 실전 에이전틱 코딩 패턴을 현재 보안 실습 흐름에 맞게 재구성합니다.



이 그림은 오늘의 핵심 구조를 파일·실행·산출물 관점으로 연결합니다.



Mini Lab → Main Lab 연결

작은 실험으로 개념을 먼저 체감하고, 바로 보안 실습으로 확장합니다.

Mini Lab · Risk Card Sorting Game

Severity
KEV
EPSS
Control Module

Main Lab · Threat Intelligence 점수화

Threat Context
Risk Score
Priority Table
Decision Support

연결 문장

직관 정렬과 계산 정렬을 비교한 뒤, CVE/KEV/EPSS 기반 risk_score_table.md로 확장합니다.

Mini Lab 실행 · Risk Card Sorting Game

한 줄 명령으로 실행하고, 산출물을 확인합니다.

왜 하는가

High가 항상 1순위가 아니라는 것을 카드 정렬 게임으로 확인합니다.

```
python course/mini_labs/day04_risk_game/run_risk_sorting_game.py
```

관련 파일

```
course/mini_labs/day04_risk_game/  
run_risk_sorting_game.py  
risk_cards.json  
risk_formula.md
```

생성 산출물

```
reports/risk_sorting_game.md
```

확인 포인트

직관 순위 vs 계산 순위
KEV/EPSS 영향
도메인 맥락 반영

DDD · Risk Score · Decision Support

부가 개념을 키워드가 아니라 실습·산출물·검증과 연결합니다.

DDD 관점

취약점은 기술 라벨만으로 평가하지 않습니다.
차량 제어 모듈 같은 도메인 맥락이 우선순위를 바꿉니다.

Risk Score

$\text{severity} \times \text{KEV} \times \text{EPSS} \times \text{exposure}$ 로 우선순위를
정렬합니다.
점수는 권고입니다.

Decision Support

점수는 자동 결정이 아닙니다.
사람의 판단과 HITL 승인으로 최종 대응을 확정합니다.

왜 외부 위협 정보가 필요한가

내부 발견만으로는 우선순위를 정할 수 없습니다. 같은 'High'라도 실제 위험은 천차만별입니다.

①

발견은 많다

어제 파이프라인은 수십 건을 찾습니다. 사람이 전부 동시에 고칠 수는 없습니다. 순서가 필요합니다.

②

라벨만으로 부족

심각도 라벨(High/Medium)은 '이론적 위험'입니다. 실제로 공격에 쓰이는지는 외부 정보와야 합니다.

③

맥락이 순위를 정한다

실제 악용 사례(KEV)와 악용 확률(EPSS)을 결합하면, '지금 가장 위험한 것'이 드러납니다.

본 과정의 모든 위협 정보 조회는 교육용 오프라인 피드를 씁니다. 외부 실시간 조회나 공격 행위는 일절 하지 않습니다(방어적 점검).

CVE · KEV · EPSS — 세 가지 신호

위협 정보는 세 신호로 압축됩니다. 각각 다른 질문에 답합니다.

① CVE	② KEV	③ EPSS
'무엇인가' 공격 취약점의 표준 식별 번호	'실제 악용되나' 실제 공격 확인 목록. 등재 = 강력한 신호	'악용 확률은' 30일 내 악용 확률 (0~1)

조합 원칙: KEV에 등재되면 무조건 상위, 미등재면 EPSS와 심각도로 순서를 정합니다.

위험도 점수 네 요소의 곱

발견을 하나의 숫자로 줄여 정렬할 수 있게 합니다. 점수식의 각 항이 무엇을 뜻하는지 이해합니다.

심각도 기본 가중치
Critical 10 · High 7 · Medium 4 · Low 2. 발견 자체의 이론적 무게입니다.

×KEV 실제 악용 가중
V KEV 등재면 1.6배. '이론'이 아니라 '실제로 쓰이는' 취약점을 끌어올립니다.

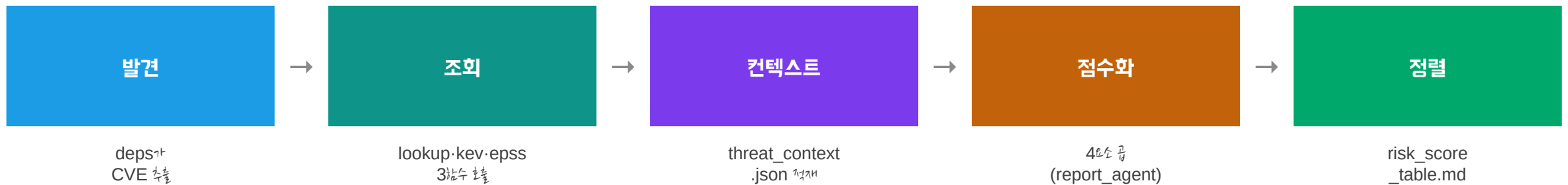
×EPSS 악용 확률 반영
SS (0.5 + EPSS). 악용 확률이 높을수록 점수가 커집니다.

× 맥락 가중
노출면 차량 제어 모듈(control/)이면 1.3배. 같은 취약점도 어디 있느냐에 따라 위험이 다릅니다.

위험도 = 심각도 × KEV가중 × (0.5+EPSS) × 노출면. 이 점수는 '권고'이며, 최종 대응 결정은 사람(HITL)의 몫입니다.

그림으로 보는 위험도 산출 발견에서 순위까지

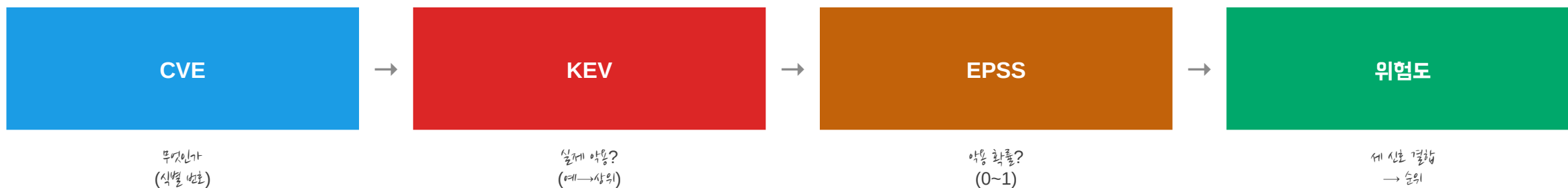
발견 하나가 우선순위 표의 한 줄이 되기까지의 전체 여정을 한 장으로 묶습니다.



이 다섯 칸이 오늘 실습·산출물 확인의 전부입니다. 각 칸이 어느 파일에서 일어나는지(threat_tools → threat_agent → report_agent) 짚으며 진행합니다.

그림으로 보는 CVE·KEV·EPSS 관계 식별에서 우선순위로

세 신호가 따로 노는 것이 아니라, 식별(CVE)에서 출발해 두 판단 신호(KEV·EPSS)로 우선순위를 만드는 관계를 봅니다.



CVE는 '누구인지'를 알려주는 이름표, KEV·EPSS는 '얼마나 급한지'를 알려주는 두 개의 온도계입니다. 셋을 곱해 위험도가 나옵니다.

라벨이 아니라 맥락이 순위를 바꾼다

오늘의 핵심 통찰입니다. 같은 심각도라도 위험 정보가 순위를 뒤집습니다.

사례

pyyaml vs SQL Injection

둘 다 High입니다. 그런데 pyyaml(CVE-2020-14343)은 KEV 등재 + EPSS 0.82로 위험도 14.8, SQL Injection은 4.2입니다.

이유

왜 역전되나

SQL Injection이 더 유명하지만, pyyaml 취약점은 '실제로 악용이 확인'되고 '악용 확률이 높기' 때문입니다.

교훈

직관의 한계

사람의 직관(유명한 것이 위험)이 아니라, 데이터(실제 악용·확률)가 우선순위를 정해야 합니다.

우선순위를 정하는 사고 순서 세 번의 질문

점수식을 외우기보다, 그 뒤에 있는 사고 순서를 익히면 어떤 발걸음 앞에서도 같은 판단을 할 수 있습니다.

Q1

실제로 적용되는가? (KEV)

예라면 다른 조건과 무관하게 상위로 올립니다. 이미 공격에 쓰이는 취약점은 '가능성'이 아니라 '현실'입니다.

Q2

곧 적용될 확률은? (EPSS)

KEV 미등재라면 EPSS로 가능합니다. 확률이 높을수록 먼저, 낮으면 심각도가 같아도 뒤로 보냅니다.

Q3

어디에 있는가? (노출면)

마지막으로 위치를 봅니다. 차량 제어 모듈 외부 노출 지점의 발견은 같은 점수라도 더 무겁게 다룹니다.

점수식은 이 세 질문을 숫자로 압축한 것일 뿐입니다. 식이 바뀌어도 질문의 순서는 변하지 않습니다.

여기까지 위협 정보 미니 점검

여기까지 이해했는지 스스로 점검해 봅니다.

☒ CVE·KEV·EPSS가 각각 답하는 질문을 안다

☒ KEV 등재가 왜 강력한 신호인지 설명한다

☒ 위협도 점수식 네 요소를 나열한다

☒ 순위 역전이란 왜 일어나는지 안다

네 가지에 답할 수 있으면 실습·산출물 확인으로 진행합니다.

실습 루프

Threat Agent 강화 + 위험도 점수화

어제까지의 파이프라인에 위협 정보를 결합하고, 발견을 우선순위 표로 바꿉니다.

오늘 실습이 하는 일 한눈에

목표, 준비물, 사용 파일, 성공 신호입니다.

목표

- threat_tools 함수 구조 확인
- 위험도 가중치 조정, 점수표 생성

준비물

- Day 3 파이프라인
- deps가 추출한 CVE

사용 파일

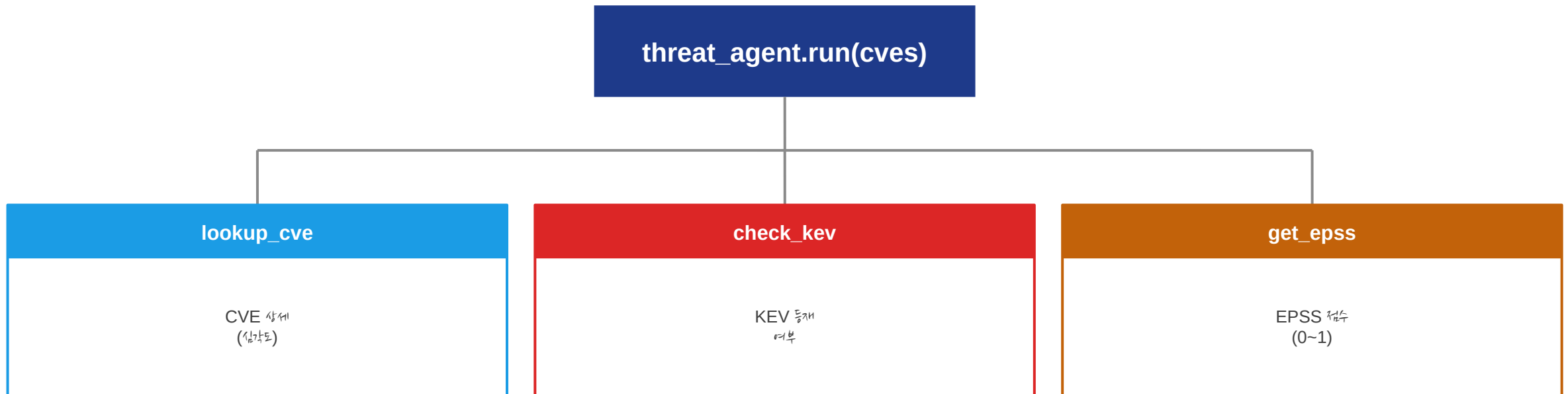
- agents/threat_agent.py
- templates/threat_tools.py

성공 신호

- threat_context.json 생성
- risk_score_table.md 정렬

threat_tools.py 세 함수 구조 확인과 가중치 조정

threat_agent.py가 이미 세 함수를 import해 사용합니다. 오늘은 그 구조를 확인하고 가중치를 조정합니다.



세 함수는 `templates/threat_tools.py`에 있고, `threat_agent`가 이미 import해 CVE별 위협 컨텍스트를 만듭니다. 코드를 새로 잇는 것이 아니라 구조를 읽고 가중치를 바꾸는 것이 오늘의 손맛입니다.

Threat Agent 실행 threat_context.json 생성

의존성 Agent가 넘긴 CVE에 대해 위협 정보를 조회합니다. 명령은 ZIP 실재 파일과 일치합니다.

단독 실행 (개념 확인)

```
python agents/threat_agent.py
```

출력

```
[Threat] 위협 컨텍스트 → reports/threat_context.json
CVE-2020-14343 High KEV등재 EPSS=0.82 PyYAML 임의 코드 실행
CVE-2023-30861 Medium - EPSS=0.12 Flask 세션 쿠키
```

threat_context.json이 오늘의 첫 산출물입니다. 이 파일이 위험도 점수 계산의 입력이 됩니다.

위협 정보를 결합한 통합 실행

--threat-intel 플래그로 파이프라인에 위협 정보를 결합하면, 위험도 점수표가 생성됩니다.

통합 실행

```
python orchestrator/supervisor_graph.py --target sample_app/ --threat-intel --report final
```

생성 산출물

reports/threat_context.json	(위협 컨텍스트)
reports/risk_score_table.md	(위험도 점수표)
reports/final_report.md	(점수표 포함 종합)

risk_score_table.md — 발견을 우선순위로

생성된 점수표입니다. 같은 High라도 KEV·EPSS·노출면에 따라 순위가 갈립니다.

순위	발견	심각도	KEV	EPSS	HITL 상태	위험도
1	pyyaml 취약 버전	High	등재	0.82	반려/보류	14.8
2	Command Injection (control/)	Critical	-	-	반려/보류	7.8
3	SQL Injection 의심	High	-	-	반려/보류	4.2
7	jinja2 취약 버전	Medium	-	0.21	확정	2.8

1순위 pyyaml은 심각도는 High지만 KEV 등재+EPSS 0.82로 위험도 14.8, 같은 High인 SQL Injection(4.2)을 크게 앞섭니다. 비대화형 실행에서는 High 이상이 기본 반려되어 HITL 상태로 구분됩니다.

워크드 예시 pyyaml 위험도를 직접 계산

점수식이 추상적으로 느껴지지 않도록, 1순위 pyyaml의 14.8이 어떻게 나오는지 순으로 따라가겠습니다.

위험도 = 심각도 × KEV가중 × (0.5+EPSS) × 노출면

```
pyyaml: High(7) × KEV(1.6) × (0.5+0.82) × 노출면(1.0)
```

계산

```
= 7 × 1.6 × 1.32 × 1.0  
= 14.78 ≈ 14.8  
비교) SQL Injection: 7 × 1.0 × (0.5+0.1) × 1.0 = 4.2
```

같은 High(7)에서 출발했지만, KEV 가중(1.6)과 높은 EPSS(0.82)가 pyyaml을 3.5배 위로 끌어올립니다.

감사 노트 '유명한 것'과 '위험한 것'은 다르다

수강생이 'SQL Injection'이 더 유명한데 왜 아래인가'라고 물을 때 답할 내용입니다.

직관 (위험할 수 있음)

- '유명한 취약점 = 위험한 취약점'
- '이름을 아는 것부터 고치자'

데이터 기반 판단

- 유명세가 아니라 실제 악용(KEV)이 기준
- 악용 확률(EPSS)이 우선순위를 정함
- 직관은 보조, 데이터가 결정

감사 팁: 이 순위 역전이 오늘의 핵심 메시지입니다. 위험 정보 결합의 가치가 바로 여기서 드러납니다.

노출면 가중 같은 취약점도 위치가 다르면

control/ 노출면 가중(1.3)이 무엇을 뜻하는지, 차량보안 맥락에서 설명합니다.

원리

어디 있느냐가 위험을 바꾼다

같은 Command Injection이라도 일반 코드보다 차량 제어 모듈(control/)에 있으면 실제 피해가 큼.

구현

노출면 1.3배

점수식에서 control/ 경로의 발견은 노출면 1.3을 곱합니다. 위치의 위험도를 숫자에 반영합니다.

확장

조직 맞춤

실무에서는 외부 노출 API, 인증 유틸 지점 등 조직별 '고위험 위치'에 더 높은 가중을 줄 수 있습니다.

가중치를 바꾸면 순위가 재정렬된다

점수식의 가중치를 직접 조정해, 우선순위가 어떻게 달라지는지 관찰합니다.

과제1 KEV 가중 강화
KEV 배수를 1.6에서 2.0으로 올려보고, KEV 등재 발견들이 더 위로 올라가는지 확인합니다.

과제2 노출면 조정
control/ 노출면 배수를 바꿔, 차량 제어 모듈 발견의 순위 변화를 봅니다.

관찰 정책의 결과
가중치는 곧 '우리 조직이 무엇을 더 위험하게 보는가'의 표현입니다. 숫자 뒤에 정책 판단이 있습니다.

여기까지 위험도 점수화 미니 점검

여기까지 이해했는지 스스로 점검해 봅니다.

☒ threat_context.json이 생성됐는지 확인했다

☒ pyyaml 위험도 14.8을 손으로 계산할 수 있다

☒ 순위 역전 사례를 설명할 수 있다

☒ 가중치 조정으로 순위가 바뀌는 것을 보았다

네 가지에 답할 수 있으면 오늘의 목표를 달성한 것입니다.

Day 4를 한 장으로 우선순위의 세 신호

오늘 다룬 내용을 세 신호로 묶습니다. 발견권을 '순서 있는 대응 계획'으로 바꾸는 열쇠입니다.

1	2	3
CVE	KEV	EPSS
무엇인가 (식별)	실제 악용되나 (확인)	악용 확률은 (예측)

세 신호를 곱으로 결합해 위험도를 만들고, 그 점수로 '무엇부터 고칠지'를 정합니다. 점수는 권고, 결정은 사람입니다.

Day 4 회고 우리 업무로 가져가기

위협 정보 결함을 부서 업무와 연결하는 회고입니다.

관찰한 것

- 레벨이 아니라 맥락이 우선순위를 정함
- KEV·EPSS가 직관을 뒷받침할 수 있음
- 점수는 권고, 결정은 사람

가져갈 질문

- 우리 조직은 무엇을 더 위협하게 보아야 하나?
- 그 판단을 가중치로 어떻게 옮길까?
- 어떤 위협 피드를 신뢰 소스로 쓸까?

여기서 적은 가중치 판단이 Day 5 통합 PoC에서 최종 리포트의 우선순위로 반영됩니다.

Day 4 — 오늘 만든 산출물 확인

오늘 저장소에 실제로 남은 것들입니다. 다음 Day의 입력적으로 누적됩니다.

01

reports/threat_context.json

CVE별 KEV·EPSS 위험 컨텍스트 (위험도 점수의 입력)

02

reports/risk_score_table.md

위험도 점수로 정렬된 우선순위 표 (HITL 상태 결정 포함)

03

updated agents/threat_agent.py

threat_tools 3함수가 연결된 강화 버전

Day 4의 세 산출물이 Day 5 통합 PoC의 final_report.md에 그대로 합류합니다.

Day 4 — 자주 나는 오류

강의장에서 가장 흔히 막히는 지점과 즉시 대응법입니다.

threat_context 비어 있음

→ Day 3의 dependency Agent가 CVE를 넘겼는지 확인

ImportError: threat_tools

→ templates/ 경로에 PYTHONPATH 확인

가중치 바뀌도 순위 동일

→ 점수식 수정 후 재실행 여부 확인

KEV/EPSS 의미 혼동

→ KEV=실제 악용 확인, EPSS=악용 확률(0~1)로 구분

Day 4 — 완료 체크리스트 · 다음 Day 연결

체크리스트로 마감하고, 내일로 이어지는 연결고리를 확인합니다.

☒ threat_tools.py 3함수 구조 확인

☒ threat_agent.py 실행 성공

☒ threat_context.json 생성

☒ KEV 등재 시 순위 상승 확인

☒ risk_score_table.md 재정렬 관찰

☒ 위험도 점수식 각 항 의미 설명

다음 Day 연결고리 → Day 5: 5일간 만든 정책·도구·조율·검증·위협정보를 하나의 파이프라인으로 통합한다

APPENDIX

부록 이론 상세 보충

본문에서 압축한 개념의 상세 설명입니다. 수업 중 질문이 나오거나, 복습 시 참조하세요.

[상세] CVE·KEV·EPSS 출처와 갱신

본문 '이론 2'의 상세판. 세 신호가 실무에서 어디서 오는지 봅니다(본 과정은 오프라인 피드 사용).

CVE

MITRE/NVD

공개 취약점에 부여되는 표준 식별자. NVD에서 상세 정보(심각도, 설명)를 제공합니다.

KEV

CISA 카탈로그

미국 CISA가 '실제 악용이 확인된' 취약점을 모은 목록. 등재는 즉시 대응 신호로 통합됩니다.

EPSS

FIRST.org

FIRST가 산출하는 30일 내 악용 확률(0~1). 매일 갱신됩니다. 통계 모델 기반 예측치입니다.

본 과정은 이 세 소스를 모사한 교육용 오프라인 피드(templates/threat_tools.py)를 씁니다. 실시간 외부 조회는 하지 않습니다.

[상세] 위험도 점수식을 곱으로 만든 이유

본문 '이론 3'의 배경. 왜 더하기가 아니라 곱하기인가.

곱의 성질

한 요소가 0에 가까우면

전체가 작아집니다. 악용 확률이 매우 낮으면 심각도가 높아도 우선순위가 내려갑니다 — 현실적입니다.

증폭

여러 요소가 동시에 높으면

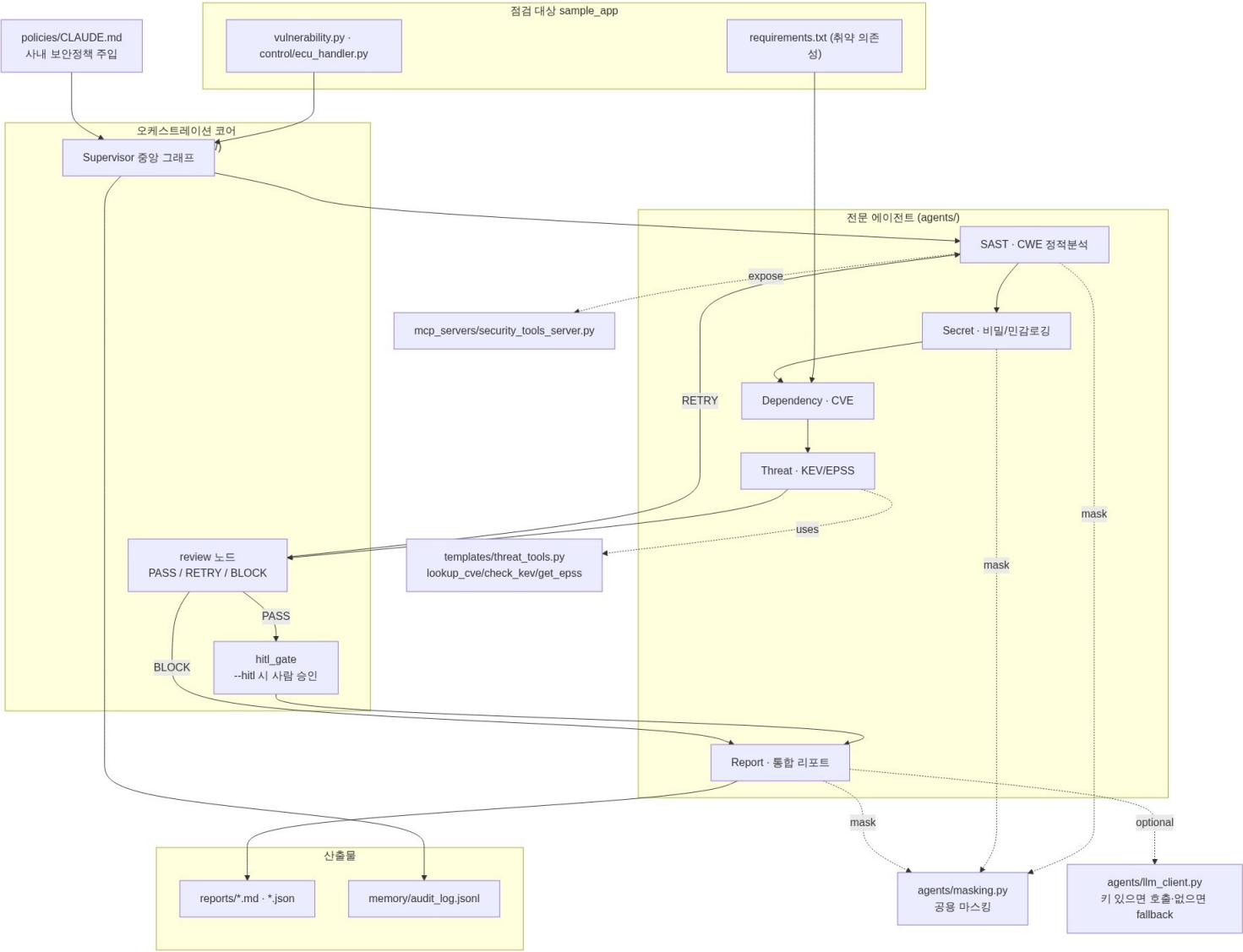
급격히 커집니다. KEV 등재 + 높은 EPSS + 제어 모듈이 결치면 위험도가 치솟습니다 — 직관과 일치합니다.

주의

점수는 도구일 뿐

점수식은 정답이 아니라 '정렬'을 위한 약속입니다. 조직마다 가중치를 조정해야 합니다.

전체 아키텍처 — Security Agent Lab



Day 4 동작 흐름 (시퀀스 다이어그램)

